

بسمه تعالی

گزارش کار سیستم عامل

Dining Philosophers

استاد
دکتر سلطانی

نویسندگان

جواد باجلان

دانشگاه شهید مدنی آذربایجان
بهار 1404

فهرست

صفحه	عنوان
۳.....	مقدمه.....
۳.....	اهمیت مسئله در سیستم عامل.....
۴.....	راه حل ها.....
۵.....	روش دایکسترا.....
۷.....	پیاده سازی.....

مسئله غذا خوردن فیلسوف‌ها (Dining Philosophers Problem) یکی از مسائل کلاسیک و مشهور در علوم کامپیوتر است که برای اولین بار توسط ادگر دایکسترا (Edsger Dijkstra) در سال ۱۹۶۵ مطرح شد. این مسئله به عنوان یک مثال آموزشی برای نمایش چالش‌های همزمانی (Concurrency) و هماهنگی (Synchronization) در سیستم‌های عامل و برنامه‌نویسی موازی استفاده می‌شود.

در این مسئله ۵ فیلسوف در یک خانه که با اعداد ۰ تا ۴ نام‌گذاری شده‌اند، به دور یک میز غذاخوری صندلی مخصوص خود را دارند و مقابل هر فیلسوف، نوع خاصی از اسپاگتی است که فقط با دو چنگال قابل خوردن است. هر فیلسوف در دور میز یا در حال تفکر است یا در حال میل کردن اسپاگتی.

مشکل اساسی آن‌جا است که در بین هر دو فیلسوف، یک چنگال قرار دارد و فقط می‌تواند از چنگال‌های مجاور خود استفاده کند. این مسئله باعث می‌شود که اگر فیلسوفی در حال غذا خوردن باشد، دو فیلسوف مجاور آن نمی‌توانند اسپاگتی را میل نمایند زیرا هر دو چنگال را نمی‌توانند در اختیار بگیرند.

اهمیت مسئله در سیستم‌عامل

این مسئله به وضوح مسائل زیر را نشان می‌دهد:

- بن‌بست (Deadlock): تمام فیلسوف‌ها ممکن است همزمان چنگال سمت چپ خود را بردارند و منتظر چنگال سمت راست بمانند.
- گرسنگی (Starvation): ممکن است فیلسوفی هرگز نتواند غذا بخورد.
- رقابت برای منابع (Resource Competition): چنگال‌ها منابع محدود هستند که فیلسوف‌ها برای آن‌ها رقابت می‌کنند.

این مسئله را می‌توان به فرآیندهای واقعی مدل‌سازی نمود به صورتی که:

- فیلسوف‌ها = فرآیندها (Processes) یا نخ‌ها (Threads)
- چنگال‌ها = منابع مشترک (Shared Resources) مانند بخشی از حافظه
- غذا خوردن = استفاده از منابع مشترک
- فکر کردن = پردازش محلی بدون استفاده از منابع مشترک

راه حل ها:

1. مرتب سازی منابع (Resource Ordering):
به هر چنگال یک شماره مرتب اختصاص دهیم و فیلسوف ها باید چنگال ها را به ترتیب صعودی بردارند

مثال:
فیلسوف ۱ چنگال ۱ را قبل از چنگال ۲ برمی دارد، در حالی که فیلسوف ۲ چنگال ۲ را قبل از چنگال ۳ برمی دارد، و به همین ترتیب.

سمافور (Semaphore):
از سمافورها برای کنترل دسترسی به چنگال ها استفاده می کنیم. سمافور mutex از برداشتن همزمان یک چنگال توسط چندین فیلسوف جلوگیری می کند. سمافورهای چنگال به فیلسوف اجازه غذا خوردن را فقط زمانی می دهند که هر دو چنگال در دسترس باشند.

2. میانجی (Waiter):
یک پیشخدمت معرفی می کنیم که دسترسی به چنگال ها را کنترل می کند. فیلسوف ها چنگال ها را از پیشخدمت درخواست می کنند.
پیشخدمت چنگال ها را فقط زمانی اعطا می کند که در دسترس باشند و منجر به بن بست نشوند.

3. مانیتور (Monitor):
از مانیتور برای مدیریت دسترسی به چنگال ها استفاده می کنیم. مانیتور انحصار متقابل را تضمین می کند، و متغیرهای شرطی حالات فیلسوف (در حال غذا خوردن، فکر کردن) را مدیریت می کنند.

4. نامتقارن (Asymmetric):
فیلسوف ها چنگال ها را به ترتیب غیر یکنواخت برمی دارند.
فیلسوف ها شماره فرد ابتدا چنگال چپ را برمی دارند، در حالی که فیلسوف ها شماره زوج ابتدا چنگال راست را برمی دارند.

در این پیاده سازی، ما از راه حل سمافور استفاده خواهیم کرد که توسط ادگر دایکسترا در EWD310¹ با عنوان Hierarchical ordering of sequential processes معرفی شده است، استفاده خواهیم کرد.

¹ Dijkstra, Edsger W. [EWD-310 \(PDF\)](#). E.W. Dijkstra Archive. Center for American History, University of Texas at Austin. ([transcription](#))

روش دایکسترا

به عقیده دایکسترا یک راه حل ابتدایی برای مسئله‌ی غذا خوردن فیلسوف‌ها این است که برای هر چنگال یک سمافور دودویی (Mutex) با مقدار اولیه ۱ (که نشان می‌دهد چنگال آزاد است) در نظر گرفته شود.

```
cycle begin think;  
    P(left-hand fork);  
    P(right-hand fork);  
    eat;  
    V(left-hand fork);  
    V(right-hand fork)  
end
```

اما مشکلی که این راه حل دارد این است که اگر فیلسوف‌ها به طور هم‌زمان چنگال سمت چپ خود را بردارد، دیگر نمی‌توانند چنگال راست خود را بردارند و در حالت گیر کرده قرار می‌گیرند (deadlock)

بنابر این می‌توان P را طوری تغییر دهیم که چنگال چپ و راست را به طور هم‌زمان و در یک عملیات بردارد

اما این روش مدنظر دایکسترا نبود و راه حل دیگری را بدون هم‌زمان‌سازی P را ارائه کرد.

دایکسترا برای هر فیلسوف یک متغیر سه حالته شامل حالت‌های در حال غذا خوردن (۲)، در حال تفکر (۰) و گرسنه (۱)، تعریف نمود به طوری که همه فیلسوف‌ها در ابتدا در حالت تفکر هستند.

همچنین محدودیت زیر را نیز معرفی کرد

$$(\forall i :: C[i] = 2 \wedge C[(i+1) \bmod 5] = 2)$$

یعنی هیچ دو فیلسوف کنار یکدیگر نمی‌توانند هر دو در حال غذا خوردن باشند.

از این فرمول نتیجه می‌گیریم که تغییر وضعیت از ۲ به ۰ (یعنی وقتی که فیلسوف از غذا خوردن به تفکر بازمی‌گردد) هرگز باعث نقض این محدودیت نمی‌شود. اما تغییر از ۰ به ۲ ممکن است این محدودیت را نقض کند. بنابراین برای این حالت، حالت گرسنگی معرفی شد.

هر فیلسوف زمانی می‌تواند از حالت ۱ (گرسنه) به حالت ۲ (غذا خوردن) تغییر وضعیت بدهد که

1. خودش گرسنه باشد (در وضعیت شماره ۱)

2. همسایه سمت چپ او در حال غذا خوردن (در وضعیت شماره ۲) نباشد.
3. همسایه سمت راست او در حال غذا خوردن (در وضعیت شماره ۲) نباشد.

$$(\exists K :: C[(K-1) \bmod 5] \neq 2 \wedge C[K] = 1 \wedge C[(K+1) \bmod 5] \neq 2)$$

بنابراین راه حال شامل

1. یک سمافور mutex با مقدار اولیه 1
2. یک آرایه‌ای با مقدار اولیه 0 برای تعیین حالت هر فیلسوف به عنوان $c[0..4]$
3. یک سمافور mutex برای هر فیلسوف با مقدار اولیه 0 به عنوان $prsem[0..4]$
4. تابع test

```

procedure test (integer value K);
  if  $C[(K-1) \bmod 5] \neq 2 \wedge C[K] = 1 \wedge C[(K+1) \bmod 5] \neq 2$  do
    begin
       $C[K] := 2$ ;
      V(prsem[K])
    end;

```

که وضعیت را برای امکان غذا خوردن فیلسوف k بررسی می‌کند.

5. حلقه اصلی:

```

cycle begin think;
  P (mutex);
   $C[w] := 1$ ;
  test (w);
  V(mutex);
  P(prsem[w]); eat;
  P(mutex);
   $C[w] := 0$ ;
  test  $[(w+1) \bmod 5]$ ;

```

```

        test [(w-1) mod 5];
        V(mutex)
    end

```

پیاده‌سازی

برای پیاده‌سازی روش دایکسترا برای حل مسئله غذا خوردن فیلسوف‌ها، از زبان C استفاده خواهیم کرد. ساختار داده فیلسوف‌ها به صورت

```

struct s_philo {
    uint32_t id;
    pthread_t thread;
    state_t state;
    char sem_name[32];
    sem_t *sem;
    uint8_t completed;
}

```

می‌باشد.

برای سمافورها از **named semaphore** استفاده خواهیم نمود. این سمافورها دارای نام می‌باشند و فقط با دانستن نام آن‌ها می‌توان از پروسه دیگر به آن‌ها دسترسی داشت و به صورت شبه-فایل در دسترس خواهند بود.

تابع **test** به صورت زیر پیاده شده است

```

static void
test(philo_t *philosopher)
{
    if (LEFT(philosopher)->state != EATING &&
        philosopher->state == HUNGRY &&
        RIGHT(philosopher)->state != EATING) {
        philosopher->state = EATING;
        sem_post(philosopher->sem);
    }
}

```

همچنین حلقه اصلی به سه قسمت براساس کارایی و مفهوم، به سه قسمت که شامل `take_forks`، `start` و `put_forks` تقسیم شده است.

```
static void *
start(void *args)
{
    philo_t *philosopher = (philo_t *)args;
    for (int i = 0; i < ROUND; i++) {
        think(philosopher);
        take_forks(philosopher);
        eat(philosopher);
        put_forks(philosopher);
    }

    philosopher->completed = 1;
    return NULL;
}
```

```
static void
take_forks(philo_t *philosopher)
{
    sem_wait(mutex);
    philosopher->state = HUNGRY;
    test(philosopher);
    sem_post(mutex);
    sem_wait(philosopher->sem);
}
```

```
static void
put_forks(philo_t *philosopher)
{
    sem_wait(mutex);
    philosopher->state = THINKING;
    test(LEFT(philosopher));
    test(RIGHT(philosopher));
    sem_post(mutex);
}
```


و برای ساخت نخ‌ها که همان فیلسوف‌ها هستند از pthread به روش fork-join استفاده شده است.

همچنین رابط خط فرمان ncurses جهت شیه‌سازی میز غذاخوری و فیلسوف‌ها استفاده شده است.

نحوه کامپایل و اجرا:

```
$ apt install cmake libncurses-dev
$ cmake . -B build/
$ make -C build/
$ ./build/dining_philosophers
```

```
./build/dining_philosophers

Dining Philosophers (Dijkstra approach)

[COLORS]
  THINKING
  HUNGRY
  EATING

Voltaire: HUNGRY
Plato: THINKING
Russell: EATING
Kant: HUNGRY
Descartes: EATING

      Plato
      Russell
      TABLE  Voltaire
      Kant
      Descartes
```

```
./build/dining_philosophers

Dining Philosophers (Dijkstra approach)

[COLORS]
  THINKING
  HUNGRY
  EATING

Voltaire: THINKING [DONE]
Plato: THINKING [DONE]
Russell: THINKING [DONE]
Kant: THINKING [DONE]
Descartes: THINKING [DONE]

      Plato
      Russell
      TABLE  Voltaire
      Kant
      Descartes
```